CML3113 WEEK 14

# Preparing for Practical Test 2

Mining and Petroleum Engineering

Advance revision and test-day guide
Test time: confirm with the lecturer



WHAT TO DO

# Your revision files

01 Use this slide deck and CML3113_Test2_practice.csv. Save both on your laptop.

02 Work through the examples in order in a new Colab notebook. Try each practice question before reading its answer.

03 Practice is ungraded. Keep your notebook for revision. No form submission is needed.

WHAT TO DO

# Practice and test-day files

01 BEFORE THE TEST: use CML3113_Test2_practice.csv with the worked examples in this section.

02 ON TEST DAY: the entry page provides a different notebook and engineering_data.csv.

03 The test downloads unlock only during the scheduled test window.

WHAT TO DO

# A new practice notebook in Colab

01 Open colab.research.google.com and sign in. Choose New notebook. Rename it Test revision.

02 Click Connect. Add a code cell with + Code. Run a cell with its triangle button.

03 Run cells from top to bottom. Save with File, then Save.

WHAT TO DO

# Uploading your practice CSV

01 In Colab, click the folder icon on the left. Click Upload and choose CML3113_Test2_practice.csv.

02 Wait until the filename appears in the Files panel. Keep its name unchanged.

03 If the session resets, upload the file again before running your cells.

WHAT TO DO

# What each column means

01 reading identifies one measurement. machine identifies the equipment that produced it.

02 load is the equipment load in percent. target is specific energy, measured in kWh/t.

03 ambient is the surrounding air temperature in °C. A blank cell means the value is missing.

WHAT TO DO

# Reading a CSV with pandas

01 import pandas as pd loads pandas and gives it the short name pd.

02 pd.read_csv(filename) reads the uploaded file. The filename is text, so it needs quotation marks.

03 A name stores the resulting DataFrame. print(table) displays it, and len(table) counts its rows.

# Reading the code syntax / Import and short names

= assigns a value to a name; == compares values. Parentheses pass inputs or continue an expression across lines. print displays a value; len counts entries. int and float convert to integer and decimal numeric values.

import pandas as pd loads pandas and gives it the short name pd. pd.read_csv refers to a function in that library.

# Reading a CSV

pd.read_csv("filename.csv") reads the named file into a DataFrame: a table with labelled columns and row labels. The file must exist in the working folder.

WORKED EXAMPLE

# Loading the supplied practice file

```
import pandas as pd
practice_filename = "CML3113_Test2_practice.csv"
practice_data = pd.read_csv(practice_filename)
print(practice_data)
print(len(practice_data))
```

**practice_filename: the exact name of the accompanying revision CSV.**

Compare the printed result with the explanation in the notebook.

# A separate working table / Returning plain values

table.copy() creates a working copy. Assigning columns on this copy preserves the source table, so you can compare before and after.

series.tolist() or array.tolist() returns a Python list of values, without pandas row labels. Preserve their order when pairing actual values and predictions.

# Selecting columns

table["column"] returns one labelled column, a Series. table[["a", "b"]] returns a DataFrame containing those named columns in that order.

WORKED EXAMPLE

# Keeping the source table

```
raw_data = practice_data.copy()
print(raw_data["target"].tolist())
```

HOW TO READ IT

raw_data: original CSV table; keep it unchanged so the analysis can be checked.

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Inputs, targets and held-out data

01 Features are the input columns used to predict. In this example, load is the feature and target is the measured result.

02 Training rows teach the model. Validation rows help compare models.

03 Keep final test rows untouched until the model choice is complete.

WHAT TO DO

# Selecting groups without mixing machines

01 isin(["M1"]) marks rows belonging to the listed machine.

02 .loc[mask] keeps the rows marked True. .copy() creates a separate table.

03 A machine must belong to one data partition. Otherwise its patterns may leak into evaluation.

# Selecting labelled rows / Membership in a group list

table.loc[row_mask] selects rows whose mask is True. With column labels, .loc selects named columns too. .copy() makes a separate working table after selection.

series.isin(["E1", "E2"]) returns True for entries matching either listed label. table[that_mask] keeps all observations belonging to those groups.

WORKED EXAMPLE

# A small group split

```
training_data = raw_data.loc[
    raw_data["machine"].isin(["M1"])
].copy()
validation_data = raw_data.loc[
    raw_data["machine"].isin(["M2"])
].copy()
print(training_data["target"].tolist())
print(validation_data["target"].tolist())
```

HOW TO READ IT

training_data: rows used to learn model settings; only these rows may determine imputation values.

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# A baseline before a fitted model

01 A baseline is a simple prediction used for comparison.

02 A mean baseline predicts the training target mean for every new row.

03 Never use validation targets to calculate the training mean.

# Repeating a baseline value / Mean of recorded values

[training_target_mean] is a one-item list. Multiplying that list by the number of validation rows repeats its value once per row. This is list repetition, not multiplication of the mean itself.

series.mean() adds known values and divides by their count. pandas skips missing values by default. The result has the same unit as the original column.

WORKED EXAMPLE

# Training-mean baseline

```
training_target_mean = training_data["target"].mean()
baseline_predictions = [training_target_mean] * len(
    validation_data
)
print(baseline_predictions)
```

**training_target_mean: mean target calculated from training rows only.**

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Mean absolute error and RMSE

01 Error = actual value − predicted value. Absolute error removes the sign.

02 MAE is the mean absolute error. RMSE is the square root of the mean squared error.

03 Lower is better on the same held-out rows. Both use the target’s unit.

WHAT TO DO

# Matching predictions to the correct rows

01 Pandas subtracts Series by row label. A filtered table can keep old labels such as 2 and 3.

02 reset_index(drop=True) replaces those labels with 0 and 1. pd.Series(a_list) also starts at 0.

03 Use the same row order for actual values and validation_predictions.

# Column arithmetic / Squared error and square root

Arithmetic on columns calculates one result per row. Series align by row label. Divide only where the denominator is valid and keep actual and predicted observations paired.

error ** 2 squares each error. After averaging, ** 0.5 takes the square root. This gives root mean squared error, which penalises large errors more strongly than mean absolute error.

# Aligning positions for subtraction / A labelled column

series.reset_index(drop=True) replaces old row labels with 0, 1, 2, and so on. drop=True discards the old labels. Apply it when pairing with a new prediction Series that has those positional labels.

pd.Series(values) creates a one-dimensional labelled sequence. With an array of fold errors, the default labels are 0, 1, 2 and so on.

# Rounded display / Magnitude of each error

series.round(3) returns values rounded to three decimal places. Python round(number, 3) rounds one number. Keep unrounded calculations for returned assessment answers.

series.abs() makes negative values positive while leaving nonnegative values unchanged. Averaging these magnitudes gives mean absolute error.

# Calculating baseline errors: step 1

actual_values: observed targets or labels in the order used for comparison.

predicted_values: predictions aligned with those actual values.

WORKED EXAMPLE

# Calculating baseline errors: step 1

```
actual_values = validation_data["target"].reset_index(drop=True)
predicted_values = pd.Series(baseline_predictions)
```

**actual_values: observed targets or labels in the order used for comparison.**

Compare the printed result with the explanation in the notebook.

# Compare the baseline with observed targets

**Pair each baseline prediction with its validation target. reset_index(drop=True) makes their row labels match by position before subtraction.**

The training mean must come from training rows only.

WORKED EXAMPLE

# Calculating baseline errors: step 2

```
baseline_errors = actual_values - predicted_values
mae = baseline_errors.abs().mean()
rmse = (baseline_errors**2).mean() ** 0.5
print(round(mae, 2), round(rmse, 2))
```

**actual_values: observed targets or labels in the order used for comparison.**

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Practice: compare two predictions

01 Actual values are 30 and 40. New predictions are 28 and 44.

02 Calculate the two errors, MAE and RMSE by hand.

03 Does this improve on the baseline MAE of 20?

WHAT TO DO

# Practice answer

**01** Errors: 2 and −4. Absolute errors: 2 and 4.

**02** MAE = 3 kWh/t. RMSE = √10 ≈ 3.16 kWh/t.

**03** Both errors are lower than the baseline on these same two rows.

WHAT TO DO

# Fitting and predicting with scikit-learn

01 LinearRegression learns a straight-line relationship. fit(X, y) learns from training features and targets.

02 predict(X_new) uses the fitted relationship on new feature_columns. It does not receive their targets.

03 Double brackets, [["load"]], retain a two-dimensional feature equipment_data.

# Creating linear regression / Learning from training examples

LinearRegression() creates an unfitted model. It will learn coefficients and an intercept from training examples. Creating it alone does not train it or make predictions.

model.fit(input_table, observed_targets) learns model settings from paired rows. The two inputs must describe the same observations in the same order. Call fit before predict.

# Predicting new targets

model.predict(input_table) returns one prediction per input row. Use the same feature columns and order as during training. Prediction uses learned settings; it does not refit the model.

# Standard names for model inputs

01 X_train: training input columns. y_train: the matching observed targets.

02 X_valid: validation input columns. y_valid: their observed targets, kept for evaluation.

03 Capital X usually means a two-dimensional table; lowercase y usually means one target per row.

# Separate inputs from answers

**X_train contains the input columns; y_train contains their observed targets.**

X_valid contains the same input columns for validation; y_valid holds the answers for checking predictions.

Never include target among the inputs.

WORKED EXAMPLE

# A fitted linear model: step 1

```
X_train = training_data[["load"]]
y_train = training_data["target"]
X_valid = validation_data[["load"]]
y_valid = validation_data["target"]
from sklearn.linear_model import LinearRegression
```

**X_train: input columns used for training.**

Compare the printed result with the explanation in the notebook.

# Create, then train, then predict

**Creating an estimator sets up an empty model. fit learns settings from training inputs and their targets. predict uses those learned settings on new inputs; it does not learn from the validation targets.**

WORKED EXAMPLE

# A fitted linear model: step 2

```
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
validation_predictions = linear_model.predict(X_valid)
print(validation_predictions.round(2))
```

**X_train: input columns used for training.**

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Missing values and a fair model comparison

01 An imputer learns replacement values, such as a median, from training features only.

02 A pipeline applies the same learned replacement before fitting or predicting.

03 Compare the baseline, linear model and tree on the same validation rows. Keep final test rows separate.

WHAT TO DO

# A pipeline with a median imputer

01 SimpleImputer(strategy="median") learns one median per training feature.

02 make_pipeline(imputer, model) applies the imputer before the model.

03 Calling fit on the pipeline learns both stages using training rows only.

# The model tools: step 1

feature_columns: the list of input column names supplied to the model.

WORKED EXAMPLE

# The model tools: step 1

```
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeRegressor
feature_columns = ["load", "ambient"]
```

**feature_columns: the list of input column names supplied to the model.**

Compare the printed result with the explanation in the notebook.

# Separate inputs from answers

**X_train contains the input columns; y_train contains their observed targets.**

X_valid contains the same input columns for validation; y_valid holds the answers for checking predictions.

Never include target among the inputs.

WORKED EXAMPLE

# The model tools: step 2

```
X_train = training_data[feature_columns]
X_valid = validation_data[feature_columns]
```

HOW TO READ IT

feature_columns: the list of input column names supplied to the model.

Compare the printed result with the explanation in the notebook.

# A sequence of model steps / Filling model inputs

make_pipeline(imputer, estimator) links preprocessing and prediction. fit learns the imputer and estimator on training rows. predict applies the learned steps to new rows in the same order.

SimpleImputer(strategy="median") learns a median for each input column during fit. It uses those training medians to fill gaps later. Never learn replacement values from validation or test rows.

WORKED EXAMPLE

# A linear pipeline

```
linear_pipeline = make_pipeline(
    SimpleImputer(strategy="median"), LinearRegression()
)
linear_pipeline.fit(X_train, y_train)
linear_predictions = linear_pipeline.predict(X_valid)
print(linear_predictions.round(2))
```

**linear_pipeline: median imputer followed by linear regression.**

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# A shallow decision tree

01 A regression tree divides training rows using feature thresholds and predicts a value within each resulting group.

02 max_depth limits the number of levels of splits. random_state fixes the random choices for repeatability.

03 With these tiny training data, loads above the observed range receive the larger training target.

# Creating a regression tree

DecisionTreeRegressor splits inputs into regions with different predicted values. max_depth limits the number of split levels. random_state fixes random choices so the example can be repeated.

# Compare a second kind of model

**A regression tree learns splits in the input columns. max_depth limits its split levels.**

The imputer fills missing inputs using training medians; the tree then learns from those filled inputs.

WORKED EXAMPLE

# A tree pipeline

```
tree_model = make_pipeline(
    SimpleImputer(strategy="median"),
    DecisionTreeRegressor(max_depth=2, random_state=7),
)
tree_model.fit(X_train, y_train)
tree_predictions = tree_model.predict(X_valid)
print(tree_predictions)
```

**tree_model: the tree regression pipeline being compared on the same validation rows.**

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Practice: compare the three models

01 Validation targets: 30 and 40.

02 Baseline predictions: 15, 15. Linear: 30, 40. Tree: 20, 20.

03 Calculate MAE and RMSE for each linear_model. Which performs best on these two rows?

WHAT TO DO

# Practice answer

01 Baseline: MAE 20, RMSE 20.62 kWh/t.

02 Linear: MAE 0, RMSE 0 kWh/t. Tree: MAE 15, RMSE 15.81 kWh/t.

03 Linear performs best here. Two synthetic rows do not prove performance on new equipment.

WHAT TO DO

# The test now reaches back to Week 13

### EIGHT CODING TASKS

Week 8: predictions and errors. Week 9: warnings and cost. Week 10: the split and cross-validation. Week 11: does an extra input help? Week 12: choose under limits. Week 13: ridge and neighbours.

### SIX READING CHOICES

Splits, fill values, cost, the sealed test rows, the ridge penalty and scaling. Record the letters in DECISIONS.

WORDS WE USE

# Words for the later weeks

| Word | Plain meaning | Week |
|---|---|---|
| **Fold** | One round of cross-validation: one group held back, the rest used to fit. | 10 |
| **Cross-validation** | Holding each group back in turn, then reporting the average and the spread. | 10 |
| **Ablation** | Adding or removing one input and comparing the error on the same rows. | 11 |
| **Ridge penalty** | A price on large coefficients that calms a wiggly curve. Alpha sets the price. | 13 |
| **Nearest neighbours** | Predict from the average of the closest training rows. | 13 |
| **Scaling** | Putting inputs on a common ruler so that one big number does not decide the distances. | 13 |

WORKED EXAMPLE

# Counting warnings and their cost

```
actual    = [0, 1, 1, 0, 1]
predicted = [0, 1, 0, 0, 1]
pairs = list(zip(actual, predicted))
tp = sum(a == 1 and p == 1 for a, p in pairs)
fn = sum(a == 1 and p == 0 for a, p in pairs)
fp = sum(a == 0 and p == 1 for a, p in pairs)
cost = 5 * fn + fp
print("tp", tp, "fn", fn, "fp", fp)
print("cost", cost)
```

```
tp 2 fn 1 fp 0
cost 5
```

**Label 1 means a warning. One warning was missed, so the cost is 5.**

### HOW TO READ IT

tp: warnings given and needed. fn: warnings missed. fp: false alarms.

zip pairs each actual label with its predicted label.

A missed warning costs 5 and a false alarm costs 1.

WORKED EXAMPLE

# Cross-validation by equipment group

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
table = pd.DataFrame({"group": list("AABBCC"),
    "load": [20, 40, 30, 50, 60, 80],
    "target": [10, 21, 14, 26, 29, 41]})
for held_out in "ABC":
    train = table[table["group"] != held_out]
    test = table[table["group"] == held_out]
    model = LinearRegression().fit(train[["load"]], train["target"])
    pred = model.predict(test[["load"]])
    error = mean_absolute_error(test["target"], pred)
    print(held_out, round(error, 2))
```

```
A 1.27
B 0.95
C 2.0
```

**Three groups give three passes and three errors.**

HOW TO READ IT

Each pass holds one group back and fits on the others.

A held-back group never appears in its own training rows.

The test asks for E1 to E5, then the mean and the spread of the five errors.

WORKED EXAMPLE

# Does an extra input help?

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
train = pd.DataFrame({"load": [20, 30, 40, 50, 60, 70],
    "temp": [20, 25, 22, 30, 28, 35],
    "target": [10.5, 15, 18.7, 25.2, 28.4, 35.9]})
valid = pd.DataFrame({"load": [35, 65], "temp": [24, 32],
    "target": [17.0, 32.6]})
for t in (train, valid):
    t["load_x_temp"] = t["load"] * t["temp"]
for cols in (["load", "temp"], ["load", "temp", "load_x_temp"]):
    model = LinearRegression().fit(train[cols], train["target"])
    pred = model.predict(valid[cols])
    error = mean_absolute_error(valid["target"], pred)
    print(len(cols), "inputs", round(error, 3))
```

```
2 inputs 0.336
3 inputs 0.145
```

### HOW TO READ IT

The new input is the load multiplied by the temperature.

Both models use the same training rows and the same validation rows.

The only change is one extra column.

**On this tiny file the extra input lowers the error. On the test file, report what the numbers show.**

WORKED EXAMPLE

# Choosing a model inside two limits

```
candidates = [
    {"name": "line", "validation_mae": 1.5, "latency_ms": 3},
    {"name": "tree", "validation_mae": 1.2, "latency_ms": 30},
    {"name": "knn", "validation_mae": 1.1, "latency_ms": 12},
    {"name": "mean", "validation_mae": 4.0, "latency_ms": 1}]
kept = [c for c in candidates
        if c["validation_mae"] <= 2.0
        and c["latency_ms"] <= 20]
ranked = sorted(kept, key=lambda c: (
    c["validation_mae"], c["latency_ms"], c["name"]))
print([c["name"] for c in kept])
print(ranked[0]["name"])
```

```
['line', 'knn']
knn
```

## HOW TO READ IT

kept holds the models that meet both limits.

sorted orders them by error, then speed, then name.

The first one in the sorted list is the choice.

**The tree is too slow and the mean is too inaccurate. Of the rest, knn has the smaller error.**

WORKED EXAMPLE

# Ridge and nearest neighbours: build the models (1/2)

```
import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import (
    PolynomialFeatures, StandardScaler)
from sklearn.linear_model import Ridge
from sklearn.neighbors import KNeighborsRegressor
ridge = make_pipeline(
    PolynomialFeatures(3, include_bias=False),
    StandardScaler(), Ridge(alpha=1.0))
knn = make_pipeline(
    StandardScaler(),
    KNeighborsRegressor(n_neighbors=3))
```

```
(nothing printed)
```

**Two recipes. Each one will be fitted on the training rows only.**

HOW TO READ IT

The ridge pipeline makes a cubic curve of load, scales it, then fits with a penalty.

The neighbour pipeline scales both inputs first.

Nothing is fitted yet.

WORKED EXAMPLE

# Ridge and nearest neighbours: fit and score (2/2)

```
from sklearn.metrics import mean_absolute_error
train = pd.DataFrame({
    "load": [20, 30, 40, 50, 60, 70, 80, 90],
    "temp": [22, 30, 25, 35, 28, 33, 26, 31],
    "target": [10.2, 15.8, 19.5, 26.4, 29.1, 35.7, 39.2, 45.6]})
valid = pd.DataFrame({
    "load": [35, 55, 75], "temp": [28, 30, 32],
    "target": [17.9, 28.0, 38.1]})
ridge.fit(train[["load"]], train["target"])
knn.fit(train[["load", "temp"]], train["target"])
for name, model, cols in [("ridge", ridge, ["load"]),
                          ("knn", knn, ["load", "temp"])]:
    pred = model.predict(valid[cols])
    print(name, round(mean_absolute_error(valid["target"], pred), 2))
```

```
ridge 1.16
knn 2.0
```

**Same rows, same measure. The smaller error is better.**

## HOW TO READ IT

Both models are fitted on the training rows only.

The ridge model sees load alone. The neighbour model sees load and temperature.

Both are scored on the same validation rows.

WHAT TO DO

# Test 2 checklist for Weeks 9 to 13

## I CAN

Count tp, fn and fp and work out a cost. Hold each group back in turn and report the mean and spread. Compare two models on the same rows. Choose the best model inside two limits.

## I CAN EXPLAIN

Why a very large ridge penalty can hurt. Why neighbours need scaled inputs. Why the test rows stay sealed. Why every fit uses training rows only.

WHAT TO DO

# Test 2 revision checklist

01 Explain the machine split and why the ambient median must come from training rows.

02 Rerun both pipelines. Compare their predictions with the baseline on the same validation rows.

03 Calculate MAE and RMSE and explain why a perfect result on this tiny file is not proof of generalisation.

WHAT TO DO

# Functions and returned answers

01 A function is a named calculation. def starts its definition. Indented lines belong to it.

02 An argument supplies the input. return sends the result back to the caller.

03 In the test, keep every supplied function name and calculate from its input.

# Defining and calling a function

def gives a reusable block a name and input parameters. The indented statements run when it is called. return sends a result back to the caller; print displays a result but does not return it.

WORKED EXAMPLE

# A practice function

```
def average_reading(values):
    reading_values = pd.Series(values)
    return float(reading_values.mean())
print(average_reading(raw_data["target"]))
print(average_reading(raw_data["load"]))
```

HOW TO READ IT

Use the inputs and operations defined in the preceding slides.

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Recording interpretation answers

01 Read each question and its labelled answer options. Choose the option supported by the data or code.

02 In DECISIONS, keep the question identifier unchanged and replace its blank value with your chosen option letter.

03 These letters identify answer choices. The portal assigns your test files automatically.

WORKED EXAMPLE

# A dictionary entry for a practice question

```
DECISIONS = {"practice": "B"}
print(DECISIONS["practice"])
```

HOW TO READ IT

Use the inputs and operations defined in the preceding slides.

Compare the printed result with the explanation in the notebook.

WHAT TO DO

# Test-day instructions

01 The practice section ends here. Keep your revision notebook for study.

02 The following slides explain the timed test files, individual work and submission.

03 Use the portal-issued notebook and CSV for the actual test.

TEST ACCESS

# Your test files on test day

01 Downloads unlock at the scheduled start time. Open the link or scan its QR code.

02 Enter your name, Google email and Student Registration Number.

03 Download your notebook and engineering_data.csv.

TEST FILES

## Open your test files

Direct URL (click or copy into your laptop browser):

https://iuea-test-entry.jothammartin-wambi.workers.dev/?test=CML3113%7CTEST2

WHAT TO DO

# Opening the notebook and CSV

01 In Colab: File → Open notebook → Upload. Select the downloaded notebook.

02 Save a copy in Drive. Connect the session.

03 In Files on the left, upload engineering_data.csv.

WHAT TO DO

# One notebook contains all answers

**01** Eight code tasks, each with a prompt, function name and marks.

**02** Six interpretation questions: record your chosen option letters in DECISIONS.

**03** Submit one notebook snapshot using the final button.

# How the 100 marks are allocated

01 Weeks 8 to 10: split 10, predictions 15, MAE and RMSE 5, group cross-validation 10.

02 Weeks 9, 11 and 12: classification and cost 10, feature ablation 10, model selection 10.

03 Week 13: ridge and nearest neighbours 15. Six interpretation choices 15.

WHAT TO DO

# Order of work

01 Run the supplied row filters, then split by equipment ID.

02 Fit the imputer and models on training records only.

03 Work down the eight tasks in order. Each is checked on its own, so move on if one is stuck.

WHAT TO DO

# The three-hour session

01 First 15 minutes: setup and file checks.

02 Next 120 minutes: individual coding tasks.

03 Next 25 minutes: interpretation and checking.

04 Final 20 minutes: prepare the snapshot and submit the form.

WHAT TO DO

# Before submitting

01 Restart the Colab session and run cells from the top.

02 Upload the same CSV again if the new session needs it.

03 Fix errors and run the final submission cell.

WHAT TO DO

# Submission and confirmation

01 Click Submit test at the end of your notebook.

02 Open the generated form link using the same Google account. Check your details.

03 Press Submit before the deadline. Keep the confirmation.

WHAT TO DO

# How grading works

01 The lecturer’s grader runs your submitted code on unseen data.

02 The private workbook records identity, submission time, status and marks.

03 A technical or identity problem remains pending until resolved.